BABU BANARASI DAS UNIVERSITY

SCHOOL OF COMPUTER APPLICATIONS

NoSQL PROJECT REPORT

STUDENT ACADEMIC RECORD SYSTEM

Submitted By	Gaurav Singh
Roll Number	1250258202
Section	BCADS23
Submitted To	Mr. Harendra Singh Rajpoot
Academic Session	2026–2027

A complete practical report with MongoDB dataset, CRUD operations, comparison operators, logical operators, sorting, filtering, counting and query-output screenshots.

CERTIFICATE

This is to certify that **Gaurav Singh**, Roll Number **1250258202**, Section **BCADS23**, has completed the NoSQL project titled **“Student Academic Record System”** using MongoDB under the guidance of **Mr. Harendra Singh Rajpoot** during the academic session 2026–2027.

_____ _____

Student Signature **Teacher Signature**

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher **Mr. Harendra Singh Rajpoot** for giving me the opportunity to work on this NoSQL project titled **“Student Academic Record System”**.

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to insert, retrieve, update, delete, sort, filter and count student records. I also learned how comparison operators and logical operators are used to query data.

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

TABLE OF CONTENTS

No.	Section
1	Introduction and Objectives
2	Tools / Requirements
3	Methodology
4	Student Dataset
5	MongoDB Operations and Screenshots
6	Operators Used
7	Conclusion
8	References
9	Appendix — Project Details

1. INTRODUCTION

MongoDB is a popular NoSQL database management system that stores data in the form of documents. Unlike traditional relational databases, MongoDB does not require data to be stored in fixed rows and columns.

In this project, a Student Academic Record System has been created using MongoDB. The database contains information about 50 students, including roll number, name, age, course, semester, marks and city. Different MongoDB operations are demonstrated using the supplied practical screenshots.

OBJECTIVES

• Understand the concept of NoSQL databases.
• Learn the basic features of MongoDB.
• Create and use a database and collection.
• Insert 50 student records.
• Perform CRUD operations.
• Use comparison operators such as $eq, $gt, $lt, $gte, $lte and $ne.
• Use logical operators such as $and, $or and $nor.
• Perform filtering, sorting, matching and counting.

2. TOOLS / REQUIREMENTS

• MongoDB / MongoDB Shell or MongoDB Compass
• A computer system
• Basic knowledge of MongoDB commands
• Student dataset used in this project

3. METHODOLOGY

The project follows a simple database workflow: create or switch to the College database → create/use the students collection → insert student documents → retrieve records → apply comparison and logical filters → update and delete records → sort and count documents → document the results.

4. STUDENT DATASET

The project contains 50 student documents. Each document includes Roll_number, name, age, course, semester, marks and city.

Roll	Name	Age	Course	Sem	Marks	City
1	Aarav Sharma	20	BCA	2	78	Lucknow
2	Diya Verma	19	BCA	2	85	Delhi
3	Rohan Gupta	21	BCA	2	67	Kanpur
4	Ananya Singh	20	BCA	2	92	Jaipur
5	Vivaan Mishra	19	BCA	2	56	Lucknow
6	Ishita Kapoor	20	BCA	2	74	Agra
7	Aditya Joshi	21	BCA	2	88	Delhi
8	Meera Saxena	19	BCA	2	63	Kanpur
9	Kabir Yadav	20	BCA	2	45	Lucknow
10	Sara Khan	21	BCA	2	81	Mumbai
11	Arjun Tiwari	20	BCA	2	72	Agra
12	Nisha Patel	19	BCA	2	95	Ahmedabad
13	Kunal Mehta	21	BCA	2	59	Delhi
14	Pihu Agarwal	20	BCA	2	76	Jaipur
15	Dev Rajput	19	BCA	2	83	Lucknow
16	Riya Choudhary	20	BCA	2	69	Kanpur
17	Manav Bansal	21	BCA	2	91	Delhi
18	Tanya Arora	19	BCA	2	54	Agra
19	Yash Thakur	20	BCA	2	77	Lucknow
20	Neha Saini	21	BCA	2	86	Jaipur
21	Harsh Vardhan	20	BCA	2	48	Mumbai
22	Simran Kaur	19	BCA	2	73	Delhi
23	Mohit Chauhan	21	BCA	2	89	Lucknow
24	Aditi Malhotra	20	BCA	2	61	Kanpur
25	Ritik Pandey	19	BCA	2	79	Agra

4. STUDENT DATASET (CONTINUED)

Student records 26–50:

Roll	Name	Age	Course	Sem	Marks	City
26	Sanya Roy	20	BCA	2	93	Kolkata
27	Abhishek Dubey	21	BCA	2	66	Lucknow
28	Kavya Nair	19	BCA	2	58	Delhi
29	Shivam Singh	20	BCA	2	84	Jaipur
30	Muskan Jain	21	BCA	2	71	Mumbai
31	Ankit Srivastava	20	BCA	2	97	Lucknow
32	Palak Goyal	19	BCA	2	82	Kanpur
33	Nikhil Rana	21	BCA	2	52	Delhi
34	Isha Bhatt	20	BCA	2	68	Agra
35	Samar Kapoor	19	BCA	2	75	Lucknow
36	Mahi Verma	20	BCA	2	90	Jaipur
37	Raj Malhotra	21	BCA	2	62	Mumbai
38	Sneha Yadav	19	BCA	2	87	Delhi
39	Aryan Bhatia	20	BCA	2	55	Lucknow
40	Jhanvi Gupta	21	BCA	2	80	Kolkata
41	Varun Shukla	20	BCA	2	94	Agra
42	Priya Singh	19	BCA	2	64	Lucknow
43	Raghav Jain	21	BCA	2	70	Delhi
44	Anjali Mishra	20	BCA	2	57	Kanpur
45	Sameer Khan	19	BCA	2	88	Mumbai
46	Shreya Tiwari	20	BCA	2	96	Lucknow
47	Akash Gupta	21	BCA	2	43	Agra
48	Komal Sethi	19	BCA	2	72	Jaipur
49	Rishabh Sharma	20	BCA	2	85	Delhi
50	Nandini Rao	21	BCA	2	79	Lucknow

5. INSERT 50 STUDENTS

MongoDB query and output screenshot:

```
>_MONGOSH
> use("College")
< switched to db College
> db.students.insertMany([
  {Roll_number:1,name:"Aarav Sharma",age:20,course:"BCA",semester:2,marks:78,city:"Lucknow"},
  {Roll_number:2,name:"Diya Verma",age:19,course:"BCA",semester:2,marks:85,city:"Delhi"},
  {Roll_number:3,name:"Rohan Gupta",age:21,course:"BCA",semester:2,marks:67,city:"Kanpur"},
  {Roll_number:4,name:"Ananya Singh",age:20,course:"BCA",semester:2,marks:92,city:"Jaipur"},
  {Roll_number:5,name:"Vivaan Mishra",age:19,course:"BCA",semester:2,marks:56,city:"Lucknow"},
  {Roll_number:6,name:"Ishita Kapoor",age:20,course:"BCA",semester:2,marks:74,city:"Agra"},
  {Roll_number:7,name:"Aditya Joshi",age:21,course:"BCA",semester:2,marks:88,city:"Delhi"},
  {Roll_number:8,name:"Meera Saxena",age:19,course:"BCA",semester:2,marks:63,city:"Kanpur"},
  {Roll_number:9,name:"Kabir Yadav",age:20,course:"BCA",semester:2,marks:45,city:"Lucknow"},
  {Roll_number:10,name:"Sara Khan",age:21,course:"BCA",semester:2,marks:81,city:"Mumbai"},
  {Roll_number:11,name:"Arjun Tiwari",age:20,course:"BCA",semester:2,marks:72,city:"Agra"},
  {Roll_number:12,name:"Nisha Patel",age:19,course:"BCA",semester:2,marks:95,city:"Ahmedabad"},
  {Roll_number:13,name:"Kunal Mehta",age:21,course:"BCA",semester:2,marks:59,city:"Delhi"},
  {Roll_number:14,name:"Pihu Agarwal",age:20,course:"BCA",semester:2,marks:76,city:"Jaipur"},
  {Roll_number:15,name:"Dev Rajput",age:19,course:"BCA",semester:2,marks:83,city:"Lucknow"},
  {Roll_number:16,name:"Riya Choudhary",age:20,course:"BCA",semester:2,marks:69,city:"Kanpur"},
  {Roll_number:17,name:"Manav Bansal",age:21,course:"BCA",semester:2,marks:91,city:"Delhi"},
  {Roll_number:18,name:"Tanya Arora",age:19,course:"BCA",semester:2,marks:54,city:"Agra"},
  {Roll_number:19,name:"Yash Thakur",age:20,course:"BCA",semester:2,marks:77,city:"Lucknow"},
  {Roll_number:20,name:"Neha Saini",age:21,course:"BCA",semester:2,marks:86,city:"Jaipur"},
  {Roll_number:21,name:"Harsh Vardhan",age:20,course:"BCA",semester:2,marks:48,city:"Mumbai"},
```

Explanation: Inserts the 50 student documents into the students collection.

6. READ / DISPLAY ALL STUDENTS

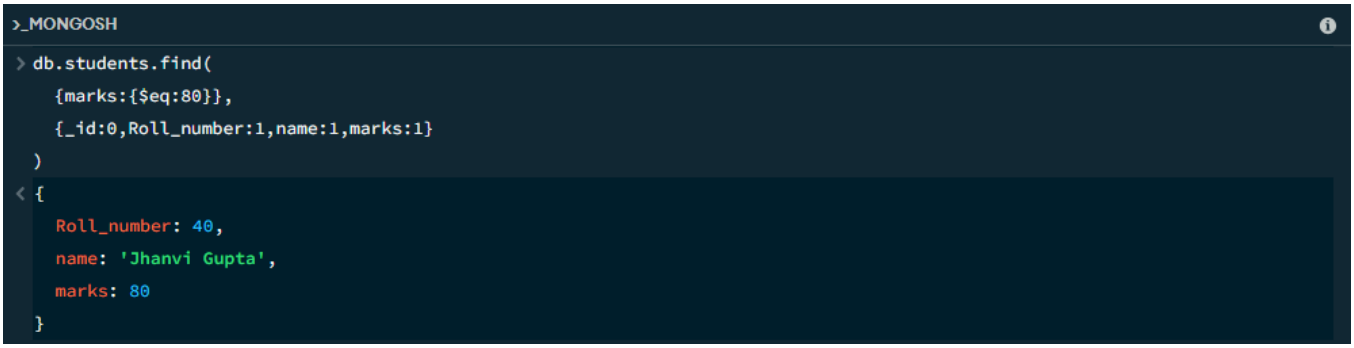
MongoDB query and output screenshot:

```
>_MONGOSH
> db.students.find({}, {_id:0})
< {
    Roll_number: 1,
    name: 'Aarav Sharma',
    age: 20,
    course: 'BCA',
    semester: 2,
    marks: 78,
    city: 'Lucknow'
  }
  {
    Roll_number: 2,
    name: 'Diya Verma',
    age: 19,
    course: 'BCA',
    semester: 2,
    marks: 85,
    city: 'Delhi'
  }
  {
    Roll_number: 3,
    name: 'Rohan Gupta',
    age: 21,
    course: 'BCA',
```

Explanation: Retrieves student documents from the students collection.

7. EQUAL TO — $eq

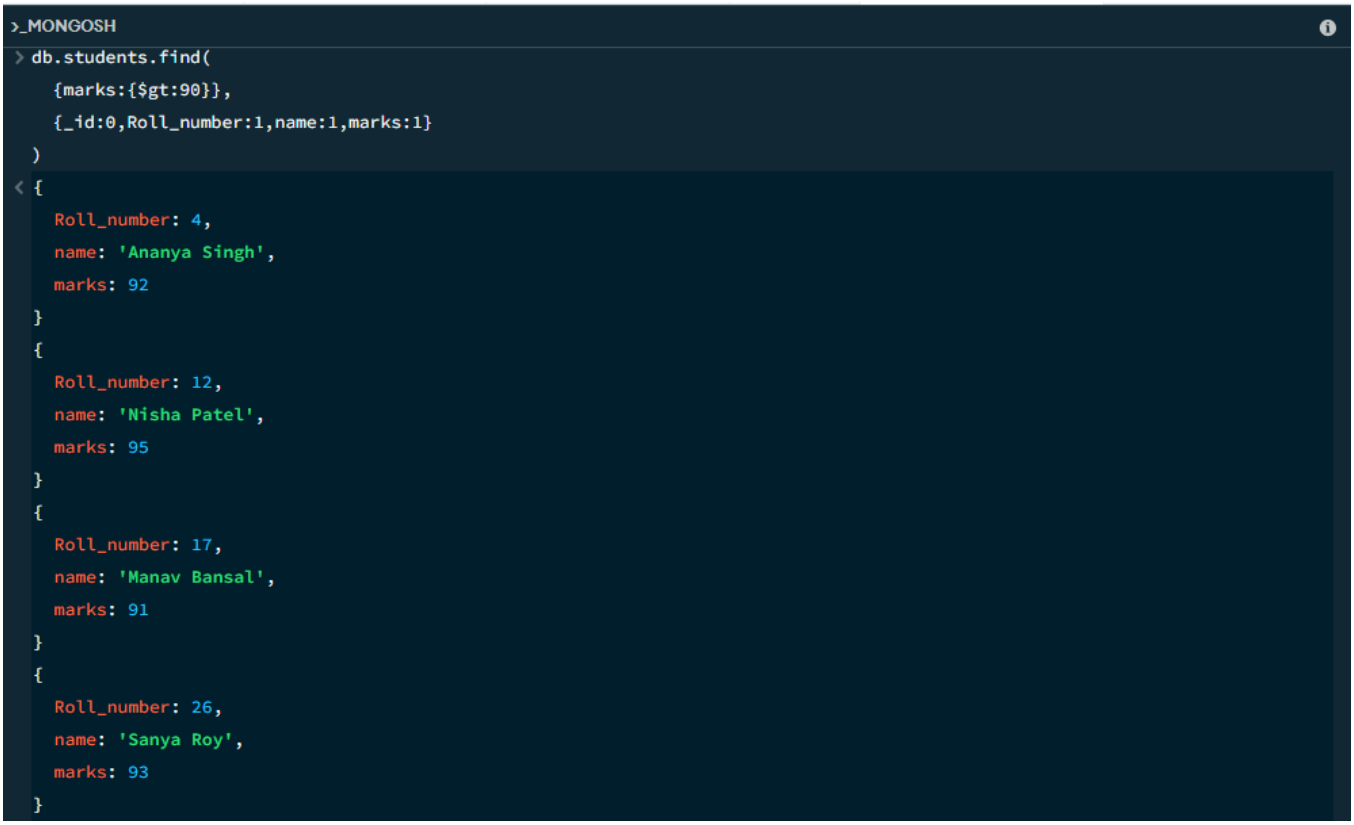
MongoDB query and output screenshot:



Explanation: Finds students whose marks are exactly equal to 80.

8. GREATER THAN — $gt

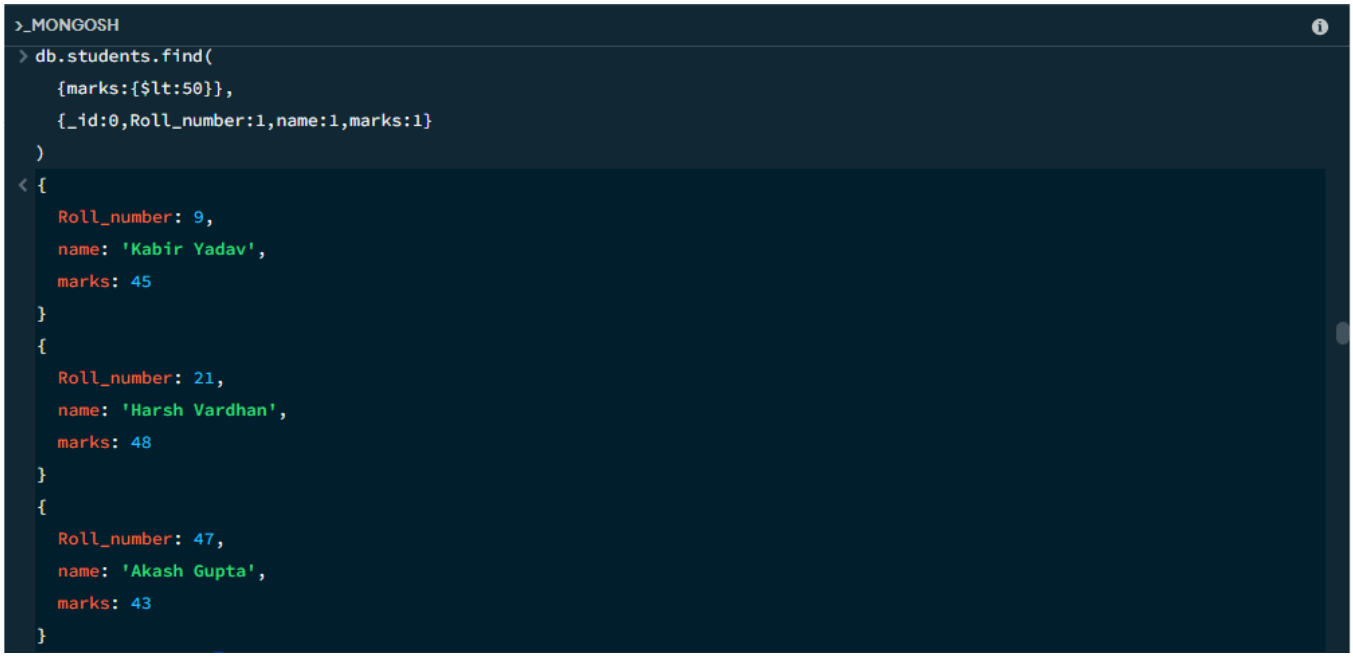
MongoDB query and output screenshot:



Explanation: Finds students whose marks are greater than 90.

9. LESS THAN — $lt

MongoDB query and output screenshot:



Explanation: Finds students whose marks are less than 50.

10. GREATER THAN OR EQUAL TO — $gte

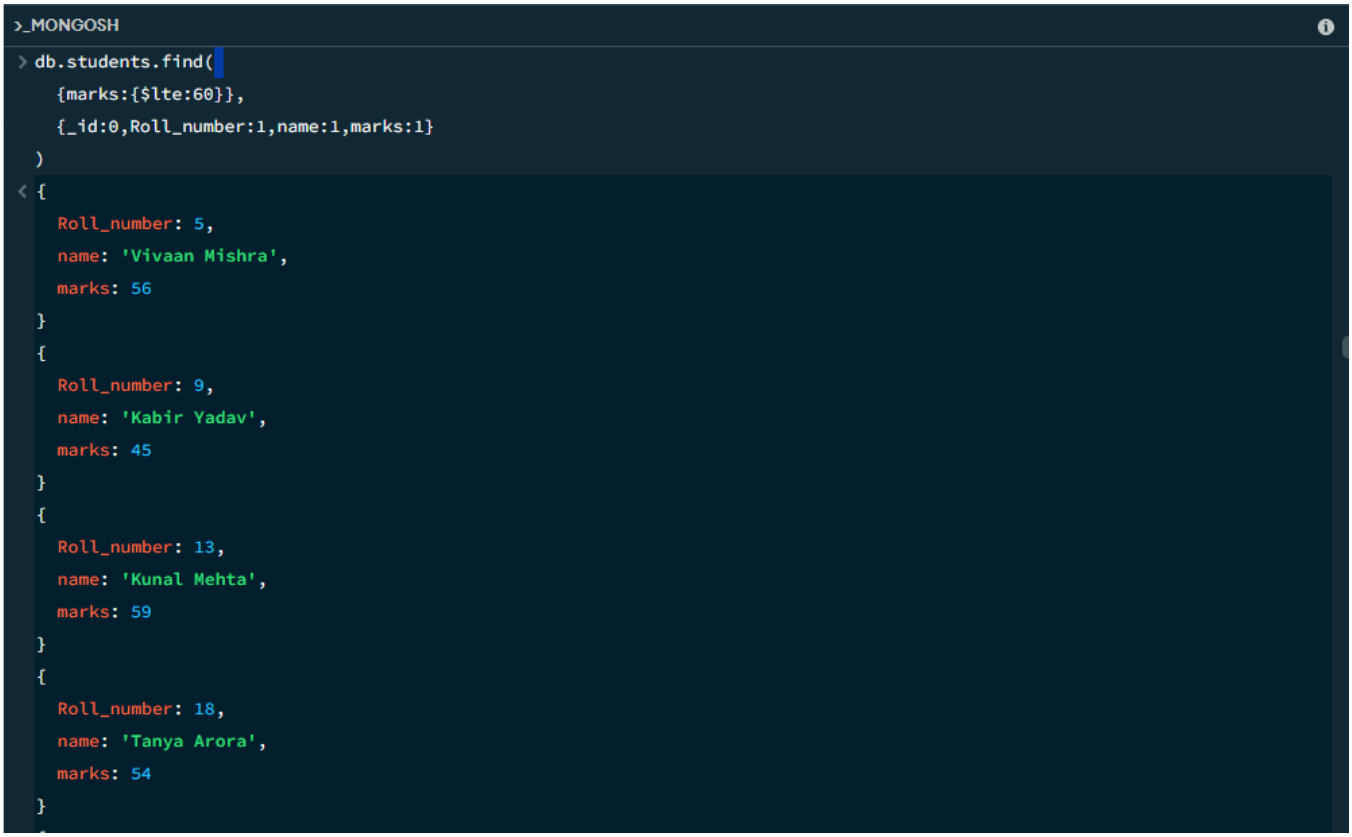
MongoDB query and output screenshot:

```
>_MONGOSH
> db.students.find(
    {marks:{$gte:85}},
    {_id:0,Roll_number:1,name:1,marks:1}
  )
< {
    Roll_number: 2,
    name: 'Diya Verma',
    marks: 85
  }
  {
    Roll_number: 4,
    name: 'Ananya Singh',
    marks: 92
  }
  {
    Roll_number: 7,
    name: 'Aditya Joshi',
    marks: 88
  }
  {
    Roll_number: 12,
    name: 'Nisha Patel',
    marks: 95
  }
```

Explanation: Finds students whose marks are greater than or equal to 85.

11. LESS THAN OR EQUAL TO — $lte

MongoDB query and output screenshot:



Explanation: Finds students whose marks are less than or equal to 60.

12. NOT EQUAL TO — $ne

MongoDB query and output screenshot:

```
>_MONGOSH
> db.students.find(
    {marks:{$ne:80}},
    {_id:0,Roll_number:1,name:1,marks:1}
  )
< {
    Roll_number: 1,
    name: 'Aarav Sharma',
    marks: 78
  }
  {
    Roll_number: 2,
    name: 'Diya Verma',
    marks: 85
  }
  {
    Roll_number: 3,
    name: 'Rohan Gupta',
    marks: 67
  }
  {
    Roll_number: 4,
    name: 'Ananya Singh',
    marks: 92
  }
```

Explanation: Finds students whose marks are not equal to 80.

13. NOT IN — $nin

MongoDB query and output screenshot:

```
>_MONGOSH
> db.students.find(
    {city:{$nin:["Lucknow","Delhi"]}},
    {_id:0,Roll_number:1,name:1,city:1}
  )
< {
    Roll_number: 3,
    name: 'Rohan Gupta',
    city: 'Kanpur'
  }
  {
    Roll_number: 4,
    name: 'Ananya Singh',
    city: 'Jaipur'
  }
  {
    Roll_number: 6,
    name: 'Ishita Kapoor',
    city: 'Agra'
  }
  {
    Roll_number: 8,
    name: 'Meera Saxena',
    city: 'Kanpur'
  }
```

Explanation: Finds students whose selected field values are not in the specified set.

14. MATCH MULTIPLE VALUES

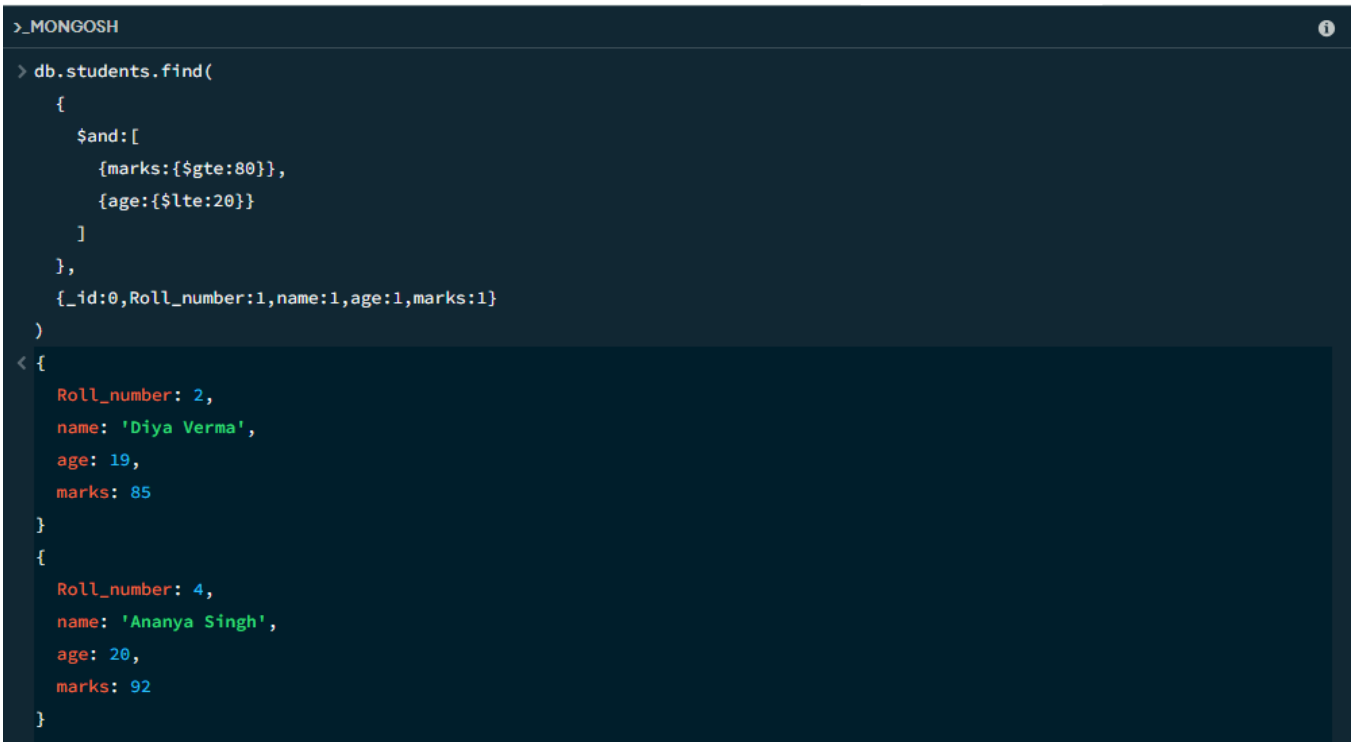
MongoDB query and output screenshot:

```
>_MONGOSH
> db.students.find(
    {city:{$in:["Lucknow","Delhi"]}},
    {_id:0,Roll_number:1,name:1,city:1}
  )
{
  Roll_number: 1,
  name: 'Aarav Sharma',
  city: 'Lucknow'
}
{
  Roll_number: 2,
  name: 'Diya Verma',
  city: 'Delhi'
}
{
  Roll_number: 5,
  name: 'Vivaan Mishra',
  city: 'Lucknow'
}
{
  Roll_number: 7,
  name: 'Aditya Joshi',
  city: 'Delhi'
}
```

Explanation: Matches records against multiple specified values.

15. AND OPERATION — $and

MongoDB query and output screenshot:



Explanation: Finds students satisfying both specified conditions.

16. OR OPERATION — $or

MongoDB query and output screenshot:

```
>_MONGOSH
> db.students.find(
    {
      $or:[
        {city:"Lucknow"},
        {city:"Delhi"}
      ]
    },
    {_id:0,Roll_number:1,name:1,city:1}
  )
{
  Roll_number: 1,
  name: 'Aarav Sharma',
  city: 'Lucknow'
}
{
  Roll_number: 2,
  name: 'Diya Verma',
  city: 'Delhi'
}
{
  Roll_number: 5,
  name: 'Vivaan Mishra',
  city: 'Lucknow'
}
```

Explanation: Finds students satisfying at least one of the specified conditions.

17. NOR OPERATION — $nor

MongoDB query and output screenshot:

```
>_MONGOSH
> db.students.find(
    {
      $nor:[
        {city:"Lucknow"},
        {city:"Delhi"}
      ]
    },
    {_id:0,Roll_number:1,name:1,city:1}
  )
{
  Roll_number: 3,
  name: 'Rohan Gupta',
  city: 'Kanpur'
}
{
  Roll_number: 4,
  name: 'Ananya Singh',
  city: 'Jaipur'
}
{
  Roll_number: 6,
  name: 'Ishita Kapoor',
  city: 'Agra'
}
```

Explanation: Finds documents that do not satisfy any of the specified conditions.

18. SORTING

MongoDB query and output screenshot:

```
>_MONGOSH
> db.students.find(
    {},
    {_id:0,Roll_number:1,name:1,marks:1}
  ).sort({marks:-1})
< {
    Roll_number: 31,
    name: 'Ankit Srivastava',
    marks: 97
  }
  {
    Roll_number: 46,
    name: 'Shreya Tiwari',
    marks: 96
  }
  {
    Roll_number: 12,
    name: 'Nisha Patel',
    marks: 95
  }
  {
    Roll_number: 41,
    name: 'Varun Shukla',
    marks: 94
  }
```

Explanation: Sorts student records according to marks.

19. UPDATE ONE OPERATION

MongoDB query and output screenshot:

```
>_MONGOSH
> db.students.updateOne(
    {Roll_number:25},
    {$set:{marks:88}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
> db.students.findOne(
    {Roll_number:25},
    {_id:0}
  )
< {
    Roll_number: 25,
    name: 'Ritik Pandey',
    age: 19,
    course: 'BCA',
    semester: 2,
    marks: 88,
    city: 'Agra'
  }
```

Explanation: Updates the marks of the student with Roll_number 25 to 88.

20. DELETE ONE OPERATION

MongoDB query and output screenshot:

```
> db.students.deleteOne(
    {Roll_number:50}
  )
< {
    acknowledged: true,
    deletedCount: 1
  }
> db.students.findOne({Roll_number:50})
< null
```

Explanation: Deletes one document matching Roll_number 50.

21. COUNT DOCUMENTS

MongoDB query and output screenshot:



Explanation: Counts the documents currently present in the collection after deletion.

22. FIND ONE OPERATION

MongoDB query and output screenshot:

```
>_MONGOSH
> db.students.findOne(
    {Roll_number:25},
    {_id:0}
  )
< {
    Roll_number: 25,
    name: 'Ritik Pandey',
    age: 19,
    course: 'BCA',
    semester: 2,
    marks: 79,
    city: 'Agra'
  }
```

Explanation: Retrieves the single student document matching Roll_number 25.

23. MONGODB OPERATORS USED

The following operators are used in the practical work represented by the screenshots supplied for this report.

Operator	Meaning	Example / Use
$eq	Equal to	marks: {$eq: 80}
$gt	Greater than	marks: {$gt: 90}
$lt	Less than	marks: {$lt: 50}
$gte	Greater than or equal to	marks: {$gte: 85}
$lte	Less than or equal to	marks: {$lte: 60}
$ne	Not equal to	marks: {$ne: 80}
$nin	Not in specified values	field: {$nin: [...]}
$and	All conditions true	$and: [...]
$or	At least one condition true	$or: [...]
$nor	None of the conditions true	$nor: [...]

24. CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Academic Record System containing 50 student records was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update and delete documents. I also learned how comparison operators such as $eq, $gt, $lt, $gte, $lte and $ne work, along with logical operators such as $and, $or and $nor. Sorting, matching, counting and single-document retrieval were also practised.

Overall, this project helped me understand how MongoDB can be used to manage and query academic records efficiently.

25. REFERENCES

1. MongoDB documentation and MongoDB Shell concepts.
2. Babu Banarasi Das University NoSQL practical/reference format supplied for this project.
3. Course notes and classroom guidance.

APPENDIX — PROJECT DETAILS

Project	Student Academic Record System
Database	College
Collection	students
Initial records	50
Student	Gaurav Singh
Roll Number	1250258202
Section	BCADS23
Teacher	Mr. Harendra Singh Rajpoot
Academic Session	2026–2027

The query panels in this report are reproduced from the screenshots supplied for the project. The student dataset is the 50-record dataset supplied with the project instructions.